

Практическая работа №3. Операторы и выражения

Дисциплина: Алгоритмизация и программирование

Инструменты: .NET SDK, VS Code + C# Dev Kit, терминал (Git Bash / встроенный терминал VS Code)

Цели работы

- Научиться пользоваться составными операторами присваивания (`+=` `-=` `*=` `/=` `%=`).
- Разобраться в разнице между `x++` и `++x`, когда результат сразу используется в выражении.
- Изучить операторы сравнения (`==` `!=` `<` `>` `<=` `>=`) и увидеть, что их результат — это `bool`.
- Изучить логические операторы (`&&` `||` `!`) и понять, что такое «короткое замыкание».
- Разобраться, в каком порядке C# вычисляет операторы, если их несколько в одном выражении.

Что вам уже знакомо. В ПР1–ПР2 вы уже писали проекты на C#, пользовались `+` `-` `*` `/`, вводом данных через `Console.ReadLine()` и преобразованием типов, коммитили и пушили в Git. Всё это здесь не объясняется заново — если что-то подзабылось, коротко загляните в ПР1/ПР2.

Важно. В этой работе вы создадите **один** проект и будете дописывать код в **один** и тот же файл **Program.cs** до самого конца. Ничего из уже написанного удалять не нужно. Благодаря этому при каждом запуске программы вы будете видеть результат сразу всех предыдущих частей — так проще заметить, если где-то раньше закралась ошибка.

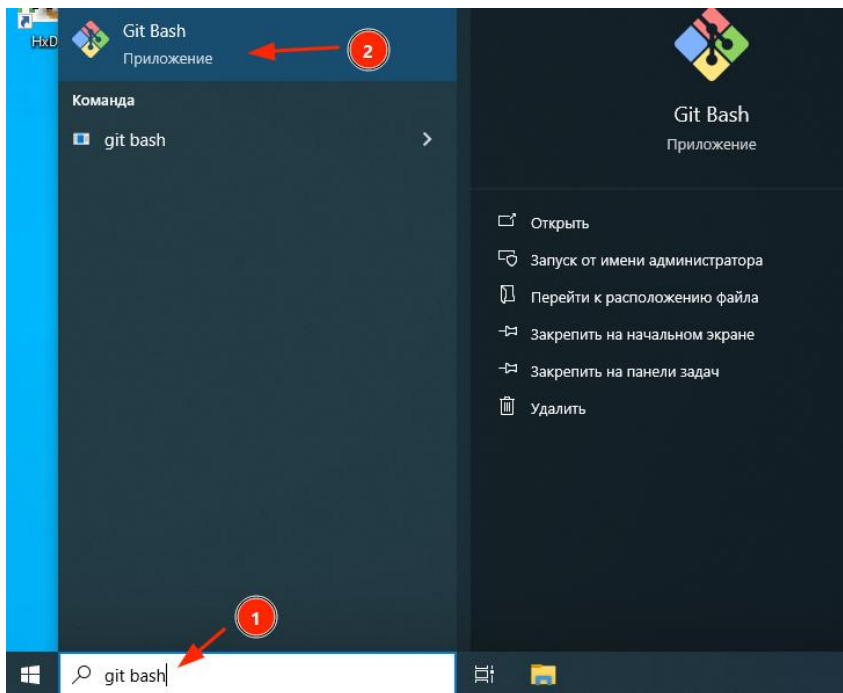
Как читать инструкцию. Перед каждым действием указано, где именно вы сейчас работаете:

В терминале VS Code — команды вводятся в нижней панели **TERMINAL**.
В файле Program.cs — код добавляется в открытый файл с кодом.

Часть 0. Подготовка проекта

Шаг 1. Перейдите в папку курса

В панели поиска Windows введите «**git bash**» и откройте программу:



В терминале **Git Bash** выполните:

cd ~/Documents

При успешном выполнении команды справа отображается путь к текущей папке:

```
leontev@denis MINGW64 ~  
$ cd ~/Documents/  
leontev@denis MINGW64 ~/Documents
```



Шаг 2. Создайте проект

В терминале **Git Bash** выполните:

dotnet new console -n Lab03-Фамилия

Например:

dotnet new console -n Lab03-Leontev (укажите свою фамилию)

Ожидаемый результат: появится новая папка Lab03-Фамилия, а в терминале — сообщение об успешном создании проекта:

```
leontev@denis MINGW64 ~/Documents  
$ dotnet new console -n Lab03-Leontev  
Шаблон "Консольное приложение" успешно создан.  
  
Идет обработка действий после создания...  
Восстановление C:\Users\leontev\Documents\Lab03-Leontev\Lab03-Leontev.csproj:  
Восстановление выполнено.
```

Шаг 3. Зайдите в проект и откройте его в VS Code

В терминале выполните по очереди:

cd Lab03-Leontev (будет ваша фамилия)

mkdir img

dotnet new gitignore

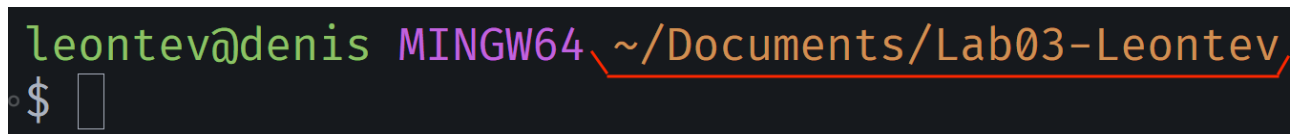
code.

Откроется новое окно VS Code, в котором видна только папка **Lab03**. Дальше работаем в этом окне.

Шаг 4. Откройте терминал внутри проекта

В **VS Code** в нижней части окна найдите вкладку **TERMINAL**. Если её нет — откройте через меню **Terminal** → **New Terminal** (или сочетание клавиш **Ctrl+J**).

Убедитесь, что терминал открылся именно в папке Lab03-Фамилия — это обычно видно в строке приглашения перед курсором:



```
leontev@denis MINGW64 ~/Documents/Lab03-Leontev
$
```

Шаг 5. Перед заливкой на GitHub

Если вы работаете за общими компьютерами, возможны проблемы из-за разных пользователей. На **каждой паре** вам нужно будет выполнять следующие пункты:

1. Проверка текущего пользователя Git

Выполните в терминале VS Code:

git config user.name

git config user.email

Значения должны совпадать с данными вашей учётной записи GitHub.

При необходимости можно задать их вручную:

git config user.name "ваш_ник"

git config user.email "ваша_почта"

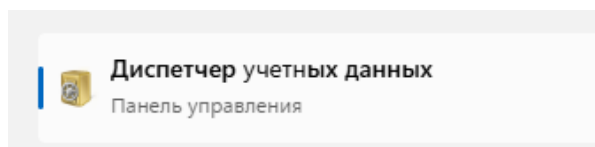
2. Вход в GitHub через браузер

Перед выполнением **git push** вы должны **выполнить вход в GitHub в браузере по умолчанию**, так как при отправке данных появится окно подтверждения авторизации.

3. Очистка старых учётных данных Windows

- Введите в поиске Windows:

Диспетчер учётных данных



- Перейдите во вкладку **Учётные данные Windows**

Администрирование учетных данных

Просмотр и удаление сохраненных сведений о входе для веб-сайтов, подключенных приложений и сетей.



Учетные данные для Интернета



Учетные данные Windows

Удалите все сохранённые записи, связанные с **GitHub**

Шаг 6. Сохраните работу в Git и создайте репозиторий на GitHub

В терминале VS Code выполните команды по очереди:

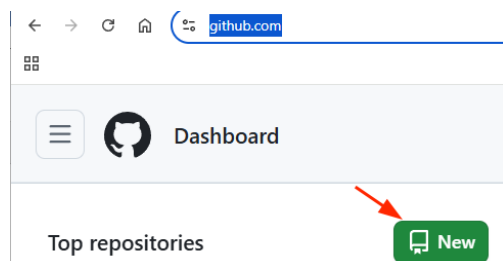
git init -b main

git add .

git commit -m "Lab03: Operators and Expressions"

1. Откройте [github](https://github.com) в браузере по умолчанию.

2. Нажмите **New**:



1 General

Great repository names are short and memorable. How about [laughing-parakeet](#)?

Owner *

Paltosik92

Repository name *

Lab03-Leontev

✓ Lab03-Leontev is available.

Description

0 / 350 characters

2 Configuration

Choose visibility *

Choose who can see and commit to this repository

Public

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Off

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore

Add license

Licenses explain how others can use your code. [About licenses](#)

No license

Create repository

3. Введите:

Название репозитория:

Lab03-Фамилия

Например:

Lab03-Ivanov

Остальные параметры

оставьте по умолчанию и

нажмите **Create repository**

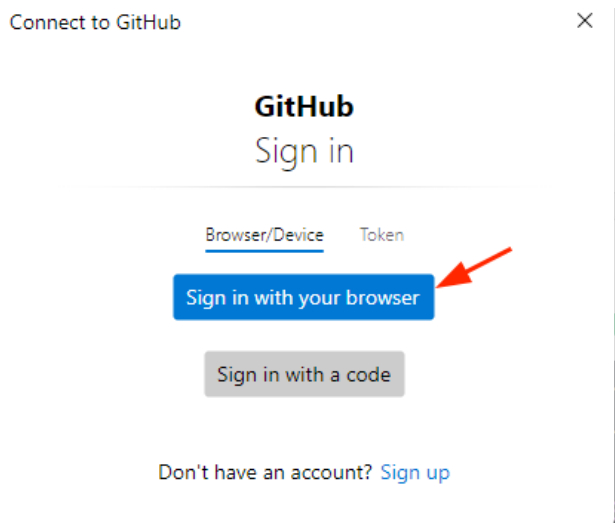
После создания **GitHub** покажет адрес репозитория и необходимые команды в самом конце для вставки в терминале:

...or push an existing repository from the command line

```
git remote add origin https://github.com/Paltosik92/Lab03-Leontev.git
git branch -M main
git push -u origin main
```

4. Во встроенном терминале VS Code введите все эти команды (для вставки в терминале Git Bash используется сочетание клавиш **Shift + Insert**)

5. При выполнении последней команды и у вас появится окно подтверждения. Нажмите на синюю кнопку **Sign in with your browser**:



При удачной загрузке вы увидите успешный вывод в терминале VS Code:

```
git push -u origin main
Enumerating objects: 31, done.
Counting objects: 100% (31/31), done.
Delta compression using up to 16 threads
Compressing objects: 100% (27/27), done.
Writing objects: 100% (31/31), 90.98 KiB | 7.58 MiB/s, done.
Total 31 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/Paltosik92/Lab03-Leontev.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Часть 1. Арифметика и составные операторы

Шпаргалка

Запись	Что означает
$x += 5$	$x = x + 5$
$x -= 3$	$x = x - 3$
$x *= 2$	$x = x * 2$
$x /= 4$	$x = x / 4$
$x \% = 4$	$x = x \% 4$

Это не новая операция, а просто более короткая запись — но пользоваться ей будете постоянно, особенно в циклах (ПР5–ПР6).

Шаг 7. Банковский счёт

Откройте файл **Program.cs** (он уже создан автоматически) и замените его содержимое на:

```
Console.WriteLine("Банковский счёт");

double balance = 1000;
Console.WriteLine($"Начальный баланс: {balance}");

balance += 500; // пополнение
Console.WriteLine($"После пополнения на 500: {balance}");

balance -= 200; // покупка
Console.WriteLine($"После покупки на 200: {balance}");

balance *= 1.05; // начисление 5% процентов
Console.WriteLine($"После начисления 5%: {balance}");

balance /= 2; // разделили счёт пополам с партнёром
Console.WriteLine($"После деления пополам: {balance}");
```

Сохраните файл (**Ctrl+S**).

Шаг 8. Запустите программу

В терминале VS Code выполните:

dotnet run

Ожидаемый результат:

```
leontev@denis MINGW64 ~/Documents/Lab03-Leontev
$ dotnet run
Банковский счёт
Начальный баланс: 1000
После пополнения на 500: 1500
После покупки на 200: 1300
После начисления 5%: 1365
После деления пополам: 682,5
```

Сверьте числа с тем, что посчитали бы вручную.

В терминале VS Code выполните:

git add .

git commit -m "Lab03: compound assignment operators"

git push

Часть 2. Инкремент и декремент

Шпаргалка

x++ и **++x** — оба увеличивают **x** на 1. Разница видна, только если результат сразу используется в выражении (например, передаётся в **Console.WriteLine**):

- **x++** — сначала отдаёт **старое** значение, потом увеличивает.
- **++x** — сначала увеличивает, потом отдаёт **новое** значение.

Если **x++** или **++x** стоит отдельной строкой — разницы нет никакой.

Шаг 9. Постфикс, префикс и практическая ловушка

В файле **Program.cs** добавьте в самый конец файла:

```
Console.WriteLine();
Console.WriteLine("Постфикс vs префикс");

int lessonNumber = 1;
Console.WriteLine($"lessonNumber++ выводит: {lessonNumber++}");
Console.WriteLine($"После этого lessonNumber = {lessonNumber}");

int weekNumber = 1;
Console.WriteLine($"++weekNumber выводит: {++weekNumber}");
Console.WriteLine($"После этого weekNumber = {weekNumber}");

Console.WriteLine();
Console.WriteLine("Практическая ловушка");

int attempts = 0;
Console.WriteLine($"Попытка №{++attempts}");
Console.WriteLine($"Попытка №{++attempts}");
Console.WriteLine($"Всего попыток: {attempts}");
```

Сохраните файл (**Ctrl+S**).

Шаг 10. Запустите программу

В терминале VS Code выполните:

dotnet run

Ожидаемый результат (внизу вывода, после уже знакомого блока про банковский счёт):

```
Постфикс vs префикс
lessonNumber++ выводит: 1
После этого lessonNumber = 2
++weekNumber выводит: 2
После этого weekNumber = 2

Практическая ловушка
Попытка №1
Попытка №2
Всего попыток: 2
```

Обратите внимание: **lessonNumber++** вывел **старое** значение (1), а **++weekNumber** — уже **новое** (2), хотя обе переменные в итоге стали равны 2.

 **СКРИНШОТ** — вывод банковского счёта + инкремента/декремента.

Сохраните в папку **img** с именем **step10Lab03_ФИО.png**

В терминале VS Code выполните:

git add .

git commit -m "Lab03: increment and decrement"

git push

Часть 3. Операторы сравнения

Шпаргалка

Оператор	Значит	Результат — всегда bool
==	равно	true / false
!=	не равно	true / false
<>	меньше / больше	true / false
<= >=	меньше или равно / больше или равно	true / false

Шаг 11. Сравниваем оценки и возраст

В файле **Program.cs** добавьте в конец файла:

```
Console.WriteLine();
Console.WriteLine("Операторы сравнения");

double myGrade = 4.6;
double passingGrade = 4.0;
int myAge = 20;
int votingAge = 18;
bool isPassing = myGrade >= passingGrade;
bool isExactAge = myAge == votingAge;
bool canVote = myAge >= votingAge;
bool isNotFailing = myGrade != 2.0;
```

```
Console.WriteLine($"Балл {myGrade} >= {passingGrade}: {isPassing}");
Console.WriteLine($"Возраст {myAge} == {votingAge}: {isExactAge}");
Console.WriteLine($"Возраст {myAge} >= {votingAge} (может голосовать): {canVote}");
Console.WriteLine($"Балл {myGrade} != 2.0 (не двойка): {isNotFailing}");
```

Сохраните файл (**Ctrl+S**).

Шаг 12. Запустите программу

В терминале VS Code выполните:

dotnet run

Ожидаемый результат (новый блок внизу):


```
Операторы сравнения
Балл 4,6 >= 4: True
Возраст 20 == 18: False
Возраст 20 >= 18 (может голосовать): True
Балл 4,6 != 2.0 (не двойка): True
```

Проверьте, что результаты совпадают с тем, что вы бы посчитали в уме.

Часть 4. Логические операторы и короткое замыкание

Шпаргалка

Оператор	Название	Когда даёт true
&&	И (AND)	когда оба условия истинны
	ИЛИ (OR)	когда хотя бы одно условие истинно
!	НЕ (NOT)	переворачивает значение на противоположное

Шаг 13. И, ИЛИ, НЕ

В файле **Program.cs** добавьте в конец файла:

```
Console.WriteLine();
Console.WriteLine("Логические операторы");

bool hasPassingGrade = true;
bool hasAttendance = false;
bool hasDebt = true;

bool canGetScholarship = hasPassingGrade && hasAttendance;
bool canRetakeExam = hasPassingGrade || hasAttendance;
bool isDebtFree = !hasDebt;
Console.WriteLine($"Может получить стипендию (оценка И посещаемость): {canGetScholarship}");
Console.WriteLine($"Может пересдать (оценка ИЛИ посещаемость): {canRetakeExam}");
Console.WriteLine($"Нет долгов: {isDebtFree}");
```

Сохраните файл (**Ctrl+S**).

Шаг 14. Запустите программу

В терминале VS Code выполните:

dotnet run

Ожидаемый результат (новый блок внизу):

```
Логические операторы
Может получить стипендию (оценка И посещаемость): False
Может пересдать (оценка ИЛИ посещаемость): True
Нет долгов: False
```

Шаг 15. Короткое замыкание своими глазами

Идея: в **a && b**, если **a** уже **false**, C# даже не проверяет **b** — результат и так **false**. Аналогично в **a || b**: если **a** уже **true**, **b** не проверяется.

Чтобы увидеть это своими глазами, нам нужен небольшой вспомогательный код. Метод (функцию) вы подробно изучите в ПР7 — сейчас просто скопируйте блок ниже, как есть, вникать в устройство **CheckAndPrint** не нужно.

В файле **Program.cs** добавьте в конец файла:

```
Console.WriteLine();
Console.WriteLine("Короткое замыкание");

bool CheckAndPrint(string label, bool value) {
    Console.WriteLine($"  Вычисляется: {label}");
    return value;
}

Console.WriteLine("Проверяем && (первый операнд false):");
bool resultAnd = CheckAndPrint("A", false) && CheckAndPrint("B", true);
Console.WriteLine($"Результат: {resultAnd}");

Console.WriteLine();
Console.WriteLine("Проверяем || (первый операнд true):");
bool resultOr = CheckAndPrint("C", true) || CheckAndPrint("D", false);
Console.WriteLine($"Результат: {resultOr}");
```

Сохраните файл (**Ctrl+S**).

Шаг 16. Запустите программу и наблюдайте

В терминале VS Code выполните:

dotnet run

Ожидаемый результат (новый блок внизу):

```
Короткое замыкание
Проверяем && (первый операнд false):
  Вычисляется: A
Результат: False

Проверяем || (первый операнд true):
  Вычисляется: C
Результат: True
```

Обратите внимание: строка **Вычисляется: B** **не появилась** — потому что **A** уже **false**, и весь **&&** заведомо **false**, дальше проверять незачем. По той же причине не появилась строка **Вычисляется: D**.

 **СКРИНШОТ** — вывод логических операторов и демонстрации короткого замыкания. Сохраните с именем **step16Lab03_ФИО.png**.

В терминале VS Code выполните:

git add .

git commit -m "Lab03: logical operators and short-circuit evaluation"

git push

Часть 5. Приоритет операций

Шпаргалка

Как и в математике, у операторов есть порядок вычисления (сверху — выполняется раньше):

```
1. Скобки ()
2. Унарные: ++ -- !
3. * / %
4. + -
5. < > <= >=
6. == !=
7. &&
8. ||
9. = += -= *= /= %=
```

Правило хорошего тона: если в выражении больше двух операторов — ставьте скобки явно, даже если по приоритету они не обязательны. Так код быстрее читается.

Шаг 17. Проверяем порядок вычисления

В файле **Program.cs** добавьте в конец файла:

```
Console.WriteLine();
Console.WriteLine("Приоритет операций");

int resultNoParens = 2 + 3 * 4;
int resultWithParens = (2 + 3) * 4;
Console.WriteLine($"2 + 3 * 4 = {resultNoParens}");
Console.WriteLine($"(2 + 3) * 4 = {resultWithParens}");

bool logicResult = 5 > 3 && 2 < 4 || false;
bool logicResultParens = (5 > 3 && 2 < 4) || false;
Console.WriteLine($"5>3 && 2<4 || false = {logicResult}");
Console.WriteLine($"(5>3 && 2<4) || false = {logicResultParens}");
```

Сохраните файл (**Ctrl+S**).

Шаг 18. Запустите программу

В терминале VS Code выполните:

dotnet run

Ожидаемый результат (новый блок внизу):

```
Приоритет операций
2 + 3 * 4          = 14
(2 + 3) * 4        = 20
5 > 3 && 2 < 4 || false = True
(5 > 3 && 2 < 4) || false = True
```

Оба логических выражения дают одинаковый результат — потому что `&&` и так имеет более высокий приоритет, чем `||`. Скобки здесь просто делают это явным для человека, который читает код.

Часть 6. Итоговая практика: приёмная комиссия

Теперь используем все операторы из этой работы вместе, в одной небольшой программе. Ввод данных и преобразование типов вы уже умеете делать — это было в ПР2.

Шаг 19. Допишите программу

В файле **Program.cs** добавьте в конец файла заготовку. В ней уже написан весь ввод данных — вам нужно дописать только четыре строки с **TODO**:

```
Console.WriteLine();
Console.WriteLine("Приёмная комиссия");

Console.Write("Введите средний балл аттестата: ");
double averageGrade = double.Parse(Console.ReadLine());

Console.Write("Введите баллы за экзамен (0-100): ");
int examScore = int.Parse(Console.ReadLine());

Console.Write("Есть льгота? (1 - да, 0 - нет): ");
int benefitInput = int.Parse(Console.ReadLine());
bool hasBenefit = (benefitInput == 1);

// TODO 1: hasGoodCertificate = true, если averageGrade >= 4.0
bool hasGoodCertificate = /* ваш код */;

// TODO 2: hasGoodExam = true, если examScore >= 60
bool hasGoodExam = /* ваш код */;

// TODO 3: isEligibleByRules = true, если
// (hasGoodCertificate И hasGoodExam) ИЛИ hasBenefit
bool isEligibleByRules = /* ваш код */;
```

```
// TODO 4: итоговый балл = средний балл * 10, а затем прибавьте баллы
// экзамена используйте составной оператор += для второго шага
double totalScore = averageGrade * 10;
/* ваш код: totalScore += ... */
Console.WriteLine();
Console.WriteLine("Результат");
Console.WriteLine($"Хороший аттестат (>= 4.0): {hasGoodCertificate}");
Console.WriteLine($"Хороший экзамен (>= 60): {hasGoodExam}");
Console.WriteLine($"Льгота: {hasBenefit}");
Console.WriteLine($"Проходит по правилам: {isEligibleByRules}");
Console.WriteLine($"Итоговый балл: {totalScore}");
```

Подсказка про `bool hasBenefit`, если непонятно: `benefitInput == 1` — это сравнение, его результат уже `true` или `false`, поэтому его можно сразу присвоить переменной типа `bool`, как и сделано в примере.

Сохраните файл (**Ctrl+S**).

Шаг 20. Запустите и проверьте на своих данных

В терминале VS Code выполните:

dotnet run

Когда программа спросит данные, введите:

```
Приёмная комиссия
Введите средний балл аттестата: 4,3
Введите баллы за экзамен (0-100): 68
Есть льгота? (1 - да, 0 - нет): 0
```

Ожидаемый результат:

```
Результат
Хороший аттестат (>= 4.0): True
Хороший экзамен (>= 60): True
Льгота: False
Проходит по правилам: True
Итоговый балл: 111
```

Если у вас получилось не так — проверьте формулы в TODO 1–4, а не код ввода.

 **СКРИНШОТ** — полный вывод программы «Приёмная комиссия» с вашими тестовыми данными. Сохраните в **img** с именем **step20Lab03_ФИО.png**.

В терминале VS Code выполните:

git add .

git commit -m "Lab03 final: admission committee calculator"

git push

Самостоятельные задания

Все задания ниже выполняются **в том же проекте** — просто дописывайте код в конец **Program.cs**

Задание 1. Чётное или нечётное — без if ★

В файле **Program.cs** допишите в конец:

1. Спросите у пользователя целое число через `Console.ReadLine()` и `int.Parse`.
2. С помощью оператора `%` определите, чётное ли оно (остаток от деления на 2 равен 0).
3. Сохраните результат в переменную `bool isEven` и выведите её на экран.

В терминале VS Code выполните:

git add .

git commit -m "Lab03 task 1: even or odd check"

git push

Задание 2. Инкремент в выражении ★★

Допишите блок кода, который на **новом примере** (не копируйте Шаг 7 один в один) показывает три разных места, где `x++` и `++x` дают разный результат в зависимости от формы. В комментариях к коду объясните, почему в каждом случае получилось именно такое значение.

В терминале VS Code выполните:

git add .

git commit -m "Lab03 task 2: increment in expressions"

git push

Задание 3. Калькулятор скидки с несколькими условиями ★★★

В файле **Program.cs** допишите в конец код, который спрашивает у пользователя:

1. Сумму покупки (`double`).
2. Есть ли карта постоянного клиента (`1/0` → `bool`, как в Шаге 17).
3. Количество товаров в чеке (`int`).

И вычисляет (**без if**, только операторами сравнения и логики) `bool eligibleForDiscount`, равное `true`, если:

- сумма покупки `>= 3000` **И** товаров в чеке `>= 3`,
- **ИЛИ** есть карта постоянного клиента.

Выведите итоговый результат и каждое промежуточное условие отдельно, чтобы было видно, из чего он сложился.

В терминале VS Code выполните:

git add .

git commit -m "Lab03 task 3: discount eligibility calculator"

git push

Контрольные вопросы

В терминале VS Code выполните:

touch README.md

Откройте **README.md**, добавьте раздел «**ПР3. Ответы на контрольные вопросы**» и ответьте:

1. Чем $x += 5$; отличается от $x = x + 5$? Есть ли разница в результате?
2. В чём разница между $x++$ и $++x$, если результат сразу используется в выражении (например, передаётся в `Console.WriteLine`)?
3. Какой тип данных всегда получается в результате оператора сравнения ($==$, $<$, $>=$ и т.д.)?
4. Что такое «короткое замыкание» в $\&\&$ и $\|\|$? Приведите свой пример, где это имеет практическое значение.
5. Расставьте приоритет для выражения $10 - 2 * 3 + 4 / 2$, посчитайте вручную, а затем проверьте кодом.
6. Почему хорошим тоном считается расставлять скобки даже там, где они не обязательны по приоритету операций?

В терминале VS Code выполните:

git add .

git commit -m "Lab03 completed"

git push